In many scenarios, the increased latency caused by a cold start is not a problem at all. The examples demonstrated in this chapter—a website health check and automated tags for EC2 instances—are not negatively affected by a cold start. To minimize cold-start times, you should keep the size of your deployment package as small as possible, provision additional memory, and use a runtime environment like JavaScript/ Node.js or Python.

However, when processing real-time data or user interactions, a cold start is undesirable. For those scenarios, you could enable provisioned concurrency for a Lambda function. With provisioned concurrency, you tell AWS to keep a certain amount of execution contexts warm, even when the Lambda function is not processing any requests. As long as the provisioned concurrency exceeds the required number of execution contexts, you will not experience cold starts. The downside is you pay $0.0000041667 for every provisioned GB per second, whether or not you use the capacity. However, you will get a discount on the cost incurred for the actual term of the Lambda function.

Another limitation is the maximum amount of memory you can provision for a Lambda function: 10,240 MB. If your Lambda function uses more memory, its execution will be terminated.

It is also important to know that CPU and networking capacity are allocated to a Lambda function based on the provisioned memory as well. So, if you are running computing- or network-intensive work within a Lambda function, increasing the provisioned memory will probably improve performance.

At the same time, the default limit for the maximum size of the compressed deployment package (zip file) is 250 MB. When